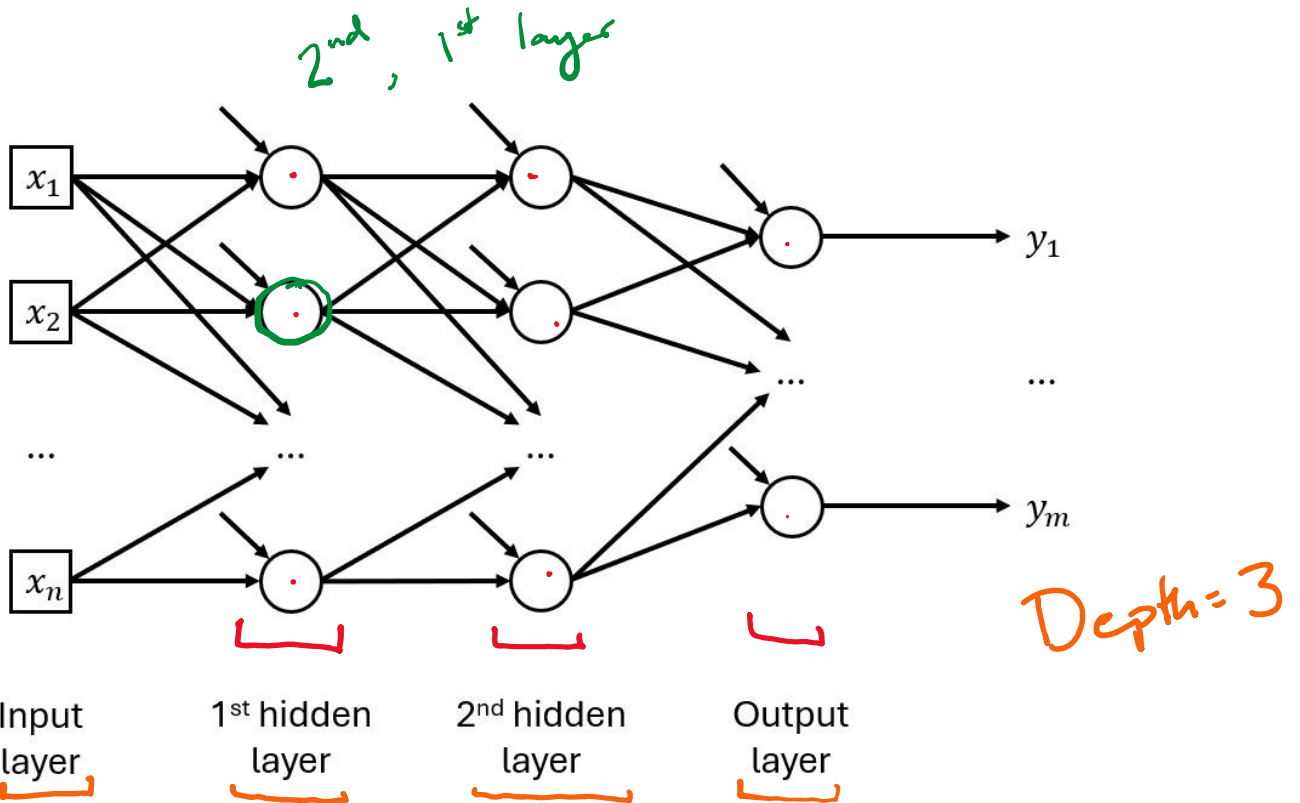


Architecture - the way neurons are arranged in a network



Depth - number of layers in a network

Width - number of neurons in a layer

Neurons are connected to every neuron in prior and subsequent layer.

Sometimes called Multi-Layer Perceptron (MLP)

Consider j^{th} neuron in the i^{th} layer...

Consider

$$X_{i+1,j} = a \left(\sum_k W_{i,j,k} \cdot X_{i,k} \right)$$

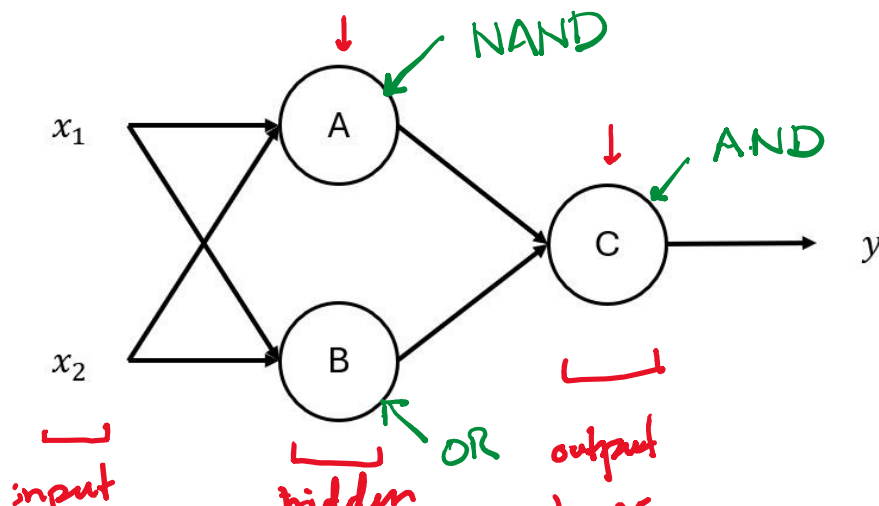
Writing as a matrix equation...

$$\vec{X}_{i+1} = a \left(W_i \vec{X}_i \right)$$

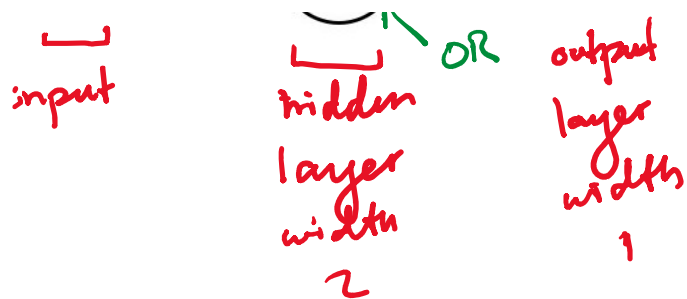
The (j,k) element of W_i weight associated with connection from k^{th} neuron in $i-1$ layer to the j^{th} neuron in i layer

Activation function is applied element-wise.

Example XOR function



Depth = 2



$$F_A(x_1, x_2) = -x_1 - x_2 + 1$$

$$F_B(x_1, x_2) = x_1 + x_2 - 1$$

$$F_C(x_A, x_B) = x_A + x_B - 2$$

Heaviside
activation

Let's write as matrix equation.

Call $\vec{x} = \begin{bmatrix} x_0=1 \\ x_1 \\ x_2 \end{bmatrix}$

$$\vec{h} = H \left(\overbrace{\begin{bmatrix} 1 & -1 & -1 \\ -1 & 1 & 1 \end{bmatrix}}^{W_1} \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \right)$$

$$\vec{o} = H \left(\underbrace{\begin{bmatrix} -2 & 1 & 1 \end{bmatrix}}_{W_2} \begin{bmatrix} 1 \\ h_1 \\ h_2 \end{bmatrix} \right)$$

Loss Functions

Assume we have input-target pairs.

Loss function measures how close predictions

Loss function measures how close predictions are to targets.

Regression tasks

$$L = \frac{1}{2} (y - \hat{y})^2$$

$\hat{y}$ - prediction

y - target

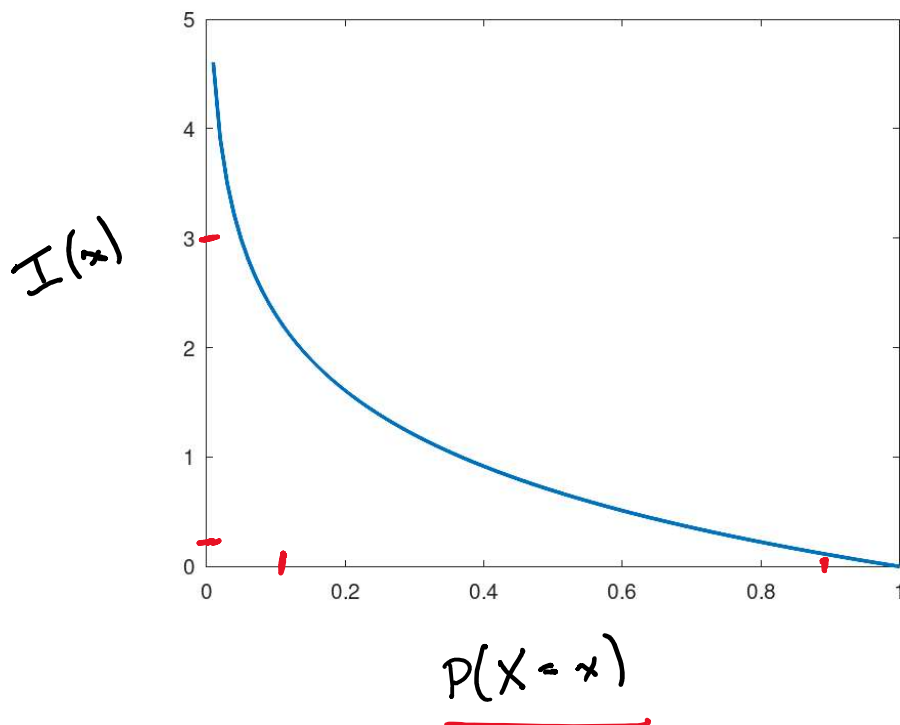
Classification tasks

~~$$L = (\hat{y} \neq y)$$~~

Information Theory

Self-Information $I(x)$ of an event $X = x$ is given by

$$I(x) = -\log P(X=x)$$



Have classifiers predict probability each class is true.

Suppose we have a classifier that predicts

$$P(\text{Class } i) = \hat{p}_i \quad \forall i$$

Now suppose the true class for our input is j .
We have $\hat{p}_j$.

The event Class j , gives us $-\log(\hat{p}_j)$ information.
This tells how erroneous model is.

Cross-Entropy Loss

Cross-Entropy Loss

$$\text{CCE Loss} = - \sum_i y_i \log(\hat{p}_i)$$

$\hat{p}_i$ - predicted probability of class i

y_i - indicates if the true class is i

This computes self-information with event Class i .

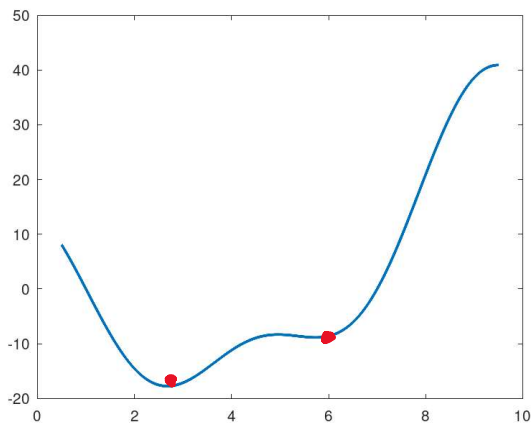
$$\text{BCE Loss} = -y \log \hat{p}_1 - (1-y) \log(1-\hat{p}_1)$$

y - true class label (0 or 1)

$\hat{p}_1$ - probability predicted for class 1

Gradient Descent

An iterative method to find the minimum of a function.



Consider a function of one variable $f(x)$
 Taylor expansion of $f(\cdot)$ at x to first order

$$f(x + \epsilon) \approx f(x) + \epsilon f'(x)$$

Observe:

1. If $f'(x) > 0$ and $\epsilon < 0$ then $f(x + \epsilon) < f(x)$
 $\Rightarrow$ If $f'(x) > 0$
 $\Rightarrow$ a small step in negative direction gives smaller value of f
2. If $f'(x) < 0$ and $\epsilon > 0$ then $f(x + \epsilon) < f(x)$
 $\Rightarrow$ If $f'(x) < 0$
 $\Rightarrow$ a small step in positive direction gives smaller value of f

Gradient Descent (Single Variable)

Given a function $f(x)$, its minimum can

Given a function $f(x)$, its minimum can be estimated by:

1. Choose initial value x and step size α .
2. Compute $f'(x)$ at x .
3. Update $x \leftarrow x - \alpha f'(x)$.
4. Repeat.